Objects passed into functions are treated differently: they are not copied even if their state changes inside the function, so any changes made will persist once the function call is complete. If this bit of trickery is causing you some confusion, take another look at Chapter 3, "Functions."

There is a drawback to using `ArrayObject`: PHP array functions don't accept them as arguments. You can export their internal data using the `ArrayObject::getArrayCopy()` method and import new data with `ArrayObject::exchangeArray()`, so the option exists to shuffle data manually in and out of an array function call. Whether or not the trade-off in functionality is worth it will be up to you!

Manipulating Arrays

Arrays are mutable: they can have new elements added or the value of existing elements changed. Using the square bracket syntax from above, new values can be assigned.

Changing Element Values

```
$myArr1 = ["Bob", "Alice"];
$myArr1[1] = "Russell";
print_r($myArr1);
```

```
Output:
Array
(
    [0] => Bob
    [1] => Russell
)
```

Logically enough, if the combination of the array variable name and key is used as a reference to an array element for obtaining its value, then it follows that we should be able to use the same method to assign a new value to the element. Using this functionality, we replace Alice with Russell.